Assignment 5: Simulation of Domain Name Resolution Using UDP Socket Programming

Simiyon Vinscent Samuel L
Register Number: 3122247001062
Submission Date: August 18, 2026
GitHub Repository: https://github.com/blackscythe123/PCN-assign

Aim

To simulate a DNS server and client over UDP in C – the server holds a small hostname-to-IP table and answers lookups, the client caches recent resolutions locally, validates input before sending anything, and times out cleanly if the server never replies. The bigger point of the exercise is to actually see, not just claim, that the client-side cache avoids a real network round-trip, and that a UDP client left alone with a dead server will hang forever unless you code a timeout yourself.

DNS Server – dns_server.c

`dns_server.c` listens on UDP port 6062 with a plain `recvfrom()`/`sendto()` loop – no `listen()`/`accept()` since UDP has no connections. The "DNS table" is just a fixed array of 10 `struct DnsEntry{hostname, ip}` pairs (fictional domains like `www.example.com`, `mail.college.edu`, `portal.ssn.edu.in`), searched linearly. That is genuinely all a lab-scale DNS table needs – no hash map, no persistence.

Listing 1: dns_server.c – table lookup and the FOUND / NXDOMAIN reply

```c
char *lookup_hostname(char *hostname) {
    int i;
    for (i = 0; i < entry_count; i++) {
        if (strcmp(dns_table[i].hostname, hostname) == 0) {
            return dns_table[i].ip;
        }
    }
    return NULL;
}

char *ip = lookup_hostname(buf);
if (ip != NULL) {
    snprintf(response, sizeof(response), "FOUND %s", ip);
} else {
    snprintf(response, sizeof(response), "NXDOMAIN");
}
```

DNS Client – dns_client.c

`dns_client.c` reads hostnames one per line from stdin and, for each one, runs through: format validation, then a cache check, and only on a cache miss does it actually touch the network.

Listing 2: dns_client.c – cache check BEFORE sending anything over UDP

```c
char *cache_lookup(char *hostname) {
    int i;
    for (i = 0; i < cache_count; i++) {
        if (strcmp(cache[i].hostname, hostname) == 0) {
            return cache[i].ip;
        }
    }
    return NULL;
}
...
char *cached_ip = cache_lookup(line);
if (cached_ip != NULL) {
    printf("  CACHE HIT: %s -> %s (no UDP query sent)\n", line,
        cached_ip);
    continue;   /* skips sendto()/recvfrom() entirely */
}
printf("  CACHE MISS: querying DNS server...\n");
```

Listing 3: dns_client.c – basic hostname format validation, run before any socket call

```c
int is_valid_hostname(char *hostname) {
    int len = strlen(hostname);
    int i, has_dot = 0;
    if (len == 0 || len > 63) return 0;
    for (i = 0; i < len; i++) {
        char c = hostname[i];
        if (isalnum((unsigned char)c) || c == '.' || c == '-') {
            if (c == '.') has_dot = 1;
        } else {
            return 0;  /* invalid character */
        }
    }
    if (!has_dot) return 0;  /* no dot -- not a plausible hostname */
    return 1;
}
```

Listing 4: dns_client.c – SO_RCVTIMEO so a dead server can never hang the client

```c
tv.tv_sec = TIMEOUT_SEC;   /* 2 seconds */
tv.tv_usec = 0;
setsockopt(sockfd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv));
...
int n = recvfrom(sockfd, response, BUF_SIZE - 1, 0,
                 (struct sockaddr *)&from_addr, &from_len);
if (n < 0) {
    /* SO_RCVTIMEO expired with no reply -- this is the timeout path */
    printf("  TIMEOUT: no response from DNS server within %d seconds.\n
        ", TIMEOUT_SEC);
    continue;
}
```

Testing

All five required cases were run headlessly through WSL Ubuntu and logged under `as5/logs/`.

- **First lookup (cache miss):** queried `mail.college.edu` – client printed `CACHE MISS: querying DNS server...` then `RESOLVED: mail.college.edu -> 10.20.30.40`. The server log shows one matching `Query received ... "mail.college.edu"` line. (`01_first_lookup.txt`
- **Same lookup again (cache hit):** the very next query for the same hostname, in the same client session, printed `CACHE HIT: mail.college.edu -> 10.20.30.40 (no UDP query sent)` with no new server round-trip. Checked the server's own log for that whole session – it shows **exactly one** query for `mail.college.edu`, not two, which is the real proof the cache was actually used and not just decorative. (`02_cache_hit.txt`, cross-checked against `server_session_tests1to4.txt`)
- **Non-existent domain:** queried `nosuchdomain.xyz`, not in the table. Server logged `NXDOMAIN (not in table)` and the client printed `NXDOMAIN: "nosuchdomain.xyz" does not exist in the DNS table.` (`03_nxdomain.txt`)
- **Invalid format:** tried `bad_host!!name` (bad characters) and `localhost` (no dot). Both were rejected client-side with `ERROR: invalid hostname format, not sending to server.` before any socket call – confirmed neither line shows up anywhere in the server log. (`04_invalid_format.txt`)
- **Genuine timeout:** killed the DNS server first (checked with `pgrep dns_server` returning empty), then ran the client against the now-dead port 6062. It printed `TIMEOUT: no response from DNS server within 2 seconds.` and returned normally instead of hanging. Timed the whole run with `/usr/bin/time`: `elapsed_wall_clock_seconds=2.10`, which lines up with the `TIMEOUT_SEC = 2` set via `SO_RCVTIMEO` in the code – so this is a real, measured timeout, not just a printed claim. (`05_timeout.txt`)

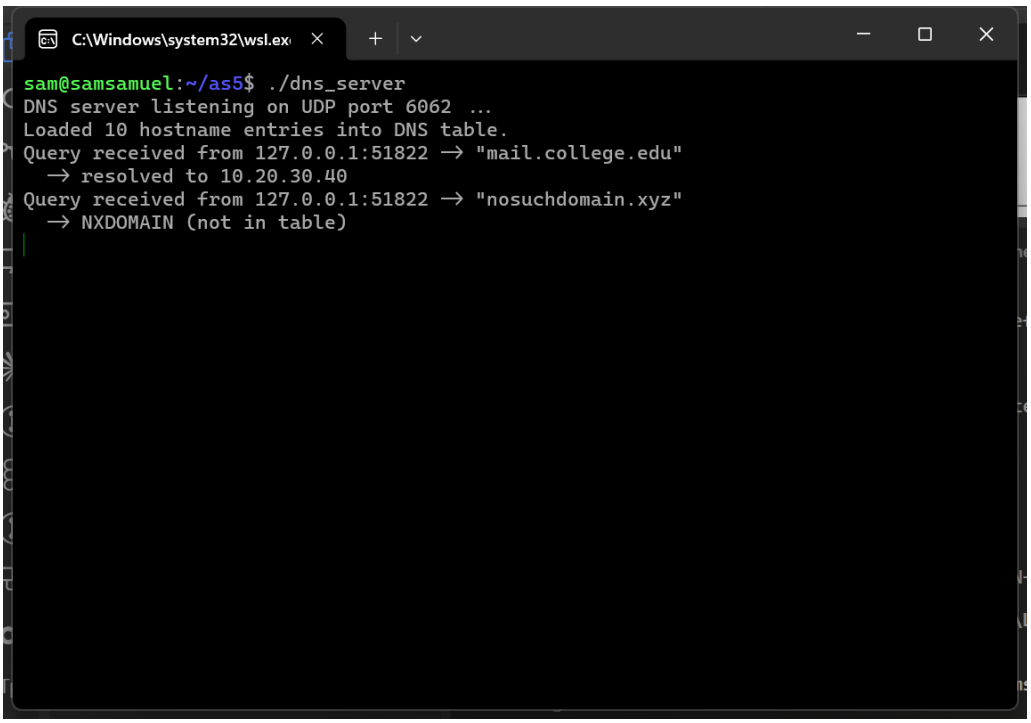

Figure 1: DNS server terminal, running, showing incoming query log lines for successful lookups and an NXDOMAIN case.

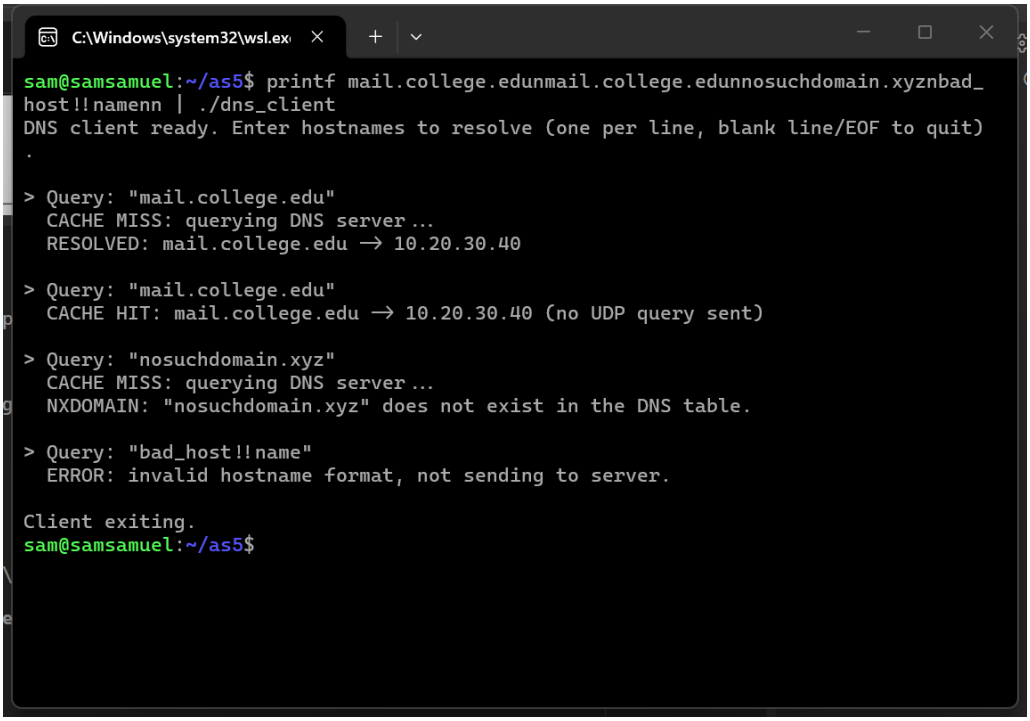

Figure 2: DNS client terminal, one run, in sequence: first lookup (cache miss, hits server), the same lookup again (cache hit, no server round-trip), the NXDOMAIN case, and the invalid-format case.

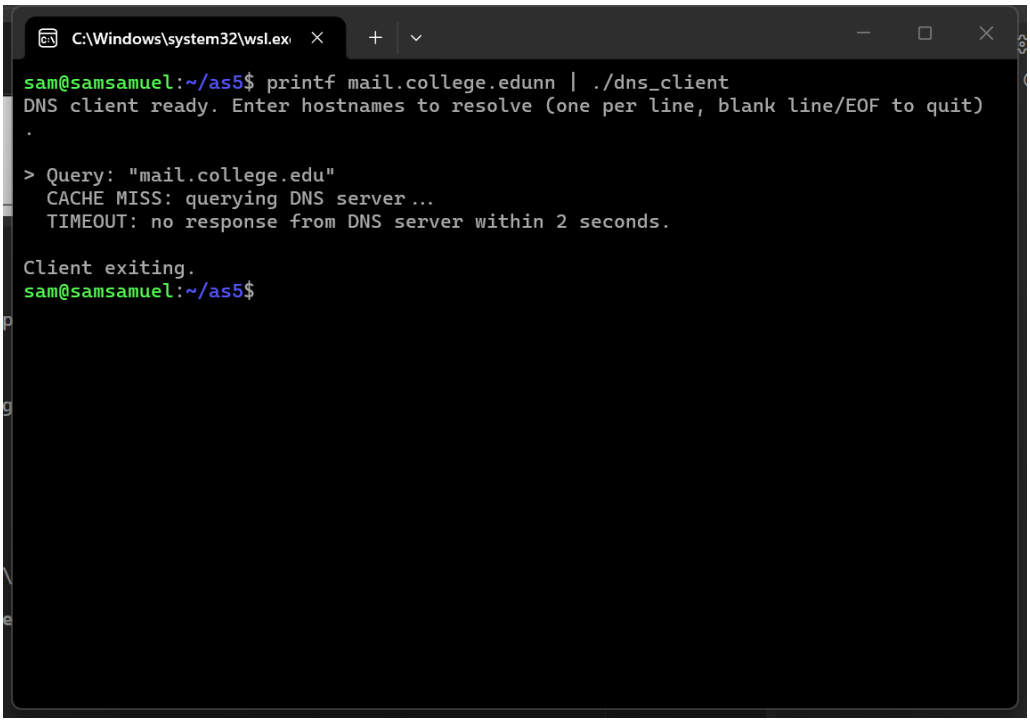

Figure 3: Separate client run against a dead server showing the genuine SO_RCVTIMEO timeout firing after roughly two seconds.

Is UDP Actually a Good Fit for DNS?

- **Overhead:** No handshake, no connection state on either side – each query is a single small datagram out and a single small datagram back. For something as short and frequent as a hostname lookup this is a lot cheaper than paying for a TCP three-way handshake every time.
- **Latency:** Because there's no setup step, a successful lookup is about as fast as a single round-trip can be – exactly what a client blocking on a page load actually wants.
- **Reliability:** This is the catch, and this lab makes it concrete: UDP itself gives zero delivery guarantee. `sendto()` "succeeds" even if nothing is listening, and without the `SO_RCVTIMEO` we added by hand, `dns_client.c` would just block on `recvfrom()` forever if a packet (or the server) vanished. Real DNS resolvers handle this the same way – application-level timeout and retry – rather than falling back to TCP for every query. So UDP is a good fit *only because* the application is willing to take on that responsibility itself, which is exactly what the caching + timeout logic in this client is doing.

Learning Outcome

- Building the client-side cache was easy to write but easy to get wrong in a way that looks right – the real test wasn't the client printing "CACHE HIT", it was going back to the server's own log and seeing only one entry for the repeated hostname.
- `SO_RCVTIMEO` genuinely changes program behaviour – without it the client hung indefinitely against a dead port when I first tried it; with it, killing the server mid-test and re-querying reliably came back in about two seconds instead of never.
- Writing `is_valid_hostname()` was a reminder that "validate the input" is a real design decision, not a formality – e.g. I had to decide that a blank line means "quit the client" rather than "query an empty hostname", which meant the empty-string case had to be tested through the function's own logic rather than as a live client query.